Automated Quality Control of Breadboard Circuits

A Comparative Analysis of Traditional Machine Learning and Convolutional Neural Networks

Aman Mishra (M00983641)
Faseeh Mohammed (M01088120)

M.Sc. Robotics
PDE4444 Machine Learning for Engineers
Coursework

Middlesex University Dubai

24 April 2026

Abstract

Manual inspection of breadboard circuits is a time-consuming quality control process that is prone to human error. This report presents an automated visual inspection system that classifies overhead circuit images as *PASS* or *FAIL* using machine learning. A raw dataset of 54 images was expanded to 864 through a systematic rotation and mirroring augmentation pipeline. Two modelling paradigms were evaluated: six traditional machine learning classifiers (SVM, Logistic Regression, KNN, Random Forest, Gradient Boosting, MLP) trained on flattened pixel vectors, and a Convolutional Neural Network (CNN) trained on full-resolution images. The baseline CNN achieved 87.02% test accuracy, and a hyperparameter-tuned variant obtained via Keras Tuner RandomSearch reached 93.75% validation accuracy. Both variants outperformed the strongest classical model, SVM, which achieved 82.08% on the test set and 86.98% under five-fold cross-validation. The report covers activation function selection, three families of optimisation algorithms, cross-validation analysis, overfitting diagnostics, and independent validation of the trained model against unseen circuit images. The final model has been integrated into a real-time OpenCV inference script, demonstrating suitability for educational laboratory and manufacturing quality control.

Contents

1 Engineering Problem Definition **4**
1.1 Engineering System Definition 4
1.2 Inputs, Outputs, and Model Justification 4
1.3 Engineering Relevance and Impact 4

2 Dataset Collection and Feature Representation **5**
2.1 Data Collection and Augmentation 5
2.2 Feature Extraction 6
2.3 Impact of Dimensionality on Model Complexity 6

3 Neural Network Design and Optimisation **7**
3.1 CNN Architecture 7
3.2 Activation Function Comparison 7
3.3 Optimisation Functions 8

4 Baseline Comparison **11**

5 Experimental Rigor **13**
5.1 Train, Validation, and Test Splits 13
5.2 Cross-Validation 13
5.3 Overfitting Analysis 14
5.4 Independent Model Validation with `analyzer.py` 17

6 Conclusion **19**

1 Engineering Problem Definition

1.1 Engineering System Definition

In electronics education and prototyping, verifying that a breadboard circuit has been assembled correctly is a recurring bottleneck. Circuits must match a master reference exactly. Any component that is missing, shifted, or incorrectly wired constitutes a failure. Manual inspection of this kind is subjective, time-consuming, and does not scale to high-throughput laboratory or manufacturing environments where hundreds of assemblies must be verified within a short window.

This project implements an automated visual quality inspection system that operates as a closed-loop pipeline. An overhead camera captures a top-down RGB image of the breadboard under test. The image is passed to a trained machine learning model, which outputs a binary classification of either **PASS** (the circuit matches the reference) or **FAIL** (a discrepancy has been detected). The result is then communicated to the operator through a visual overlay, an LED indicator, or a robotic sorting arm.

The classification goal follows a strict matching philosophy. Any deviation from the reference circuit topology, whether a missing component, an incorrect insertion row, or a wire routed along the wrong path, is treated as a failure. Partial credit is not given, as downstream electronic behaviour depends on the circuit being assembled exactly as specified.

1.2 Inputs, Outputs, and Model Justification

Two feature representations are used in this work, one for each modelling paradigm. The CNN operates on full-resolution RGB images at $224 \times 224 \times 3$, equating to 150,528 input dimensions with the 2D spatial structure preserved. The traditional machine learning pipeline downscales images to 64×64 and flattens them into a one-dimensional vector of 12,288 features, at which point all spatial layout information is discarded. The output of both pipelines is a single binary label per image: PASS (1) or FAIL (0).

A CNN is appropriate for image-based spatial reasoning because its convolutional filters learn hierarchical features such as edges, textures, and component shapes directly from the 2D image structure. This is achieved through weight sharing and pooling, without relying on the absolute pixel position of any feature. Traditional classifiers operating on flattened vectors cannot capture this spatial context. A wire shifted by a few pixels changes many feature values simultaneously, which makes generalisation fragile. The comparison between the two paradigms in this report quantifies the benefit of the inductive bias that the convolutional architecture provides.

1.3 Engineering Relevance and Impact

Automated Optical Inspection (AOI) is well established in printed circuit board (PCB) manufacturing, where machine vision systems detect soldering defects,

missing components, and misalignment. This project applies equivalent principles at the breadboard prototype level, where manual inspection is currently the norm.

The system has been deployed in several real-world configurations. The script `Scripts/analyzer.py` provides both a static image analysis mode and a live webcam inference mode using OpenCV, overlaying a real-time PASS or FAIL mask on the video feed. The script `Scripts/led_notifier.py` implements an Arduino-based LED status indicator triggered by the model output. A prior version, preserved at `Legacy/analyzer_with_robot.py`, integrates with a robotic sorting arm for automated circuit handling. Together these deployments demonstrate that the approach is viable beyond a purely academic exercise.

2 Dataset Collection and Feature Representation

2.1 Data Collection and Augmentation

The raw dataset consists of 54 manually captured overhead images, comprising 19 PASS and 35 FAIL examples, and is stored in `Dataset/breadboard_dataset/`. Images were captured under slight variations in angle and lighting to introduce natural diversity and reduce the risk of the model overfitting to a single fixed camera pose.

Neural networks typically require substantially more data to generalise well, and 54 images is far below the threshold at which a convolutional network can be reliably trained from scratch. An offline augmentation pipeline implemented in `Utils/offline_augmentation.py` was therefore applied to each original image. Each image was rotated through eight angles at 45° increments (0, 45, 90, 135, 180, 225, 270, and 315 degrees), and each rotated version was additionally mirrored horizontally, producing 16 variants per original image. Non-axis-aligned rotations such as 45° and 135° introduce triangular corner regions that contain no circuit content. These are filled with solid black to preserve semantic validity and to prevent the model from learning spurious features from the rotated corners.

The resulting augmented dataset contains $54 \times 16 = 864$ images in total, with 304 PASS and 560 FAIL examples, and is stored in `Dataset/augmented_dataset/`. The original class balance, approximately 35% PASS and 65% FAIL, is preserved because identical augmentation is applied to both classes.

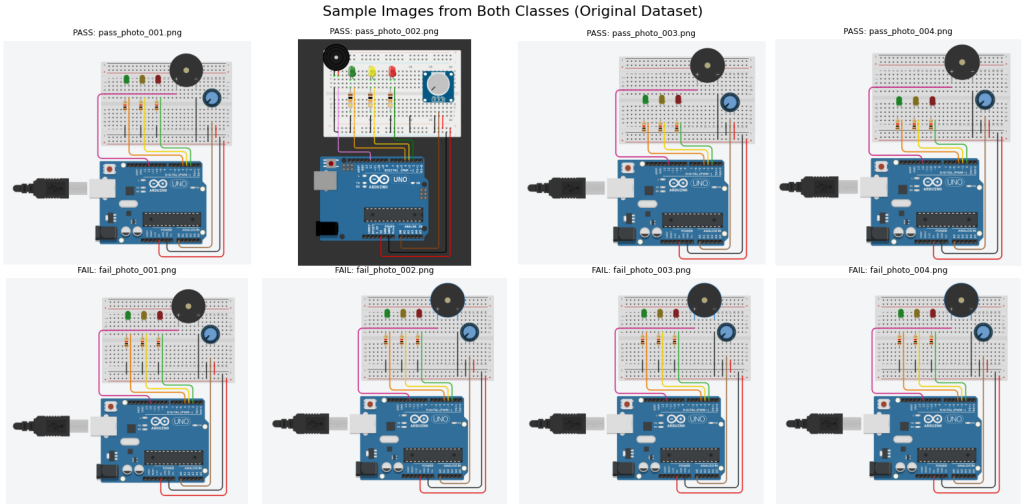

Figure 1: Sample images from the augmented dataset. Top row: PASS circuits; bottom row: FAIL circuits.

2.2 Feature Extraction

The two modelling pipelines differ substantially in their feature pipelines. Table 1 summarises the principal differences.

Table 1: Feature extraction pipeline comparison

Property	**Traditional ML**	**CNN**
Input resolution	64×64	224×224
Feature dimensionality	12,288	150,528
Spatial structure preserved	No (flattened)	Yes (2D)
Normalisation	None (raw pixel values)	Rescaling layer ($\div 255$)
Storage format	CSV file	`image_dataset_from_directory`

The flattened CSV at `Dataset/flattened_dataset/flattened_breadboards.csv` contains 864 rows and 12,289 columns (12,288 pixel features and one label column) and is generated by `Utils/image_flattener.py`. The CNN pipeline loads images directly from disk through the Keras dataset API, which handles resizing, batching, and on-the-fly shuffling.

2.3 Impact of Dimensionality on Model Complexity

The traditional ML pipeline operates in a regime where the sample-to-feature ratio is approximately 1:14. With 864 training samples and 12,288 features, the data is sparse in its high-dimensional feature space. This setting is prone to the curse of dimensionality, in which distance metrics lose discriminative power and models are likely to overfit to the training distribution. The mitigation strategies available depend on the model in use. The SVM kernel trick maps features into a higher-dimensional space where linear separation becomes feasible, with the regularisation parameter C controlling the margin width. Logistic Regression uses L_2 regularisation to penalise large weights. Random Forest and Gradient Boosting use tree depth limits and ensemble averaging to reduce variance.

For the optimiser comparison experiment in Section 3.3, PCA is applied to reduce the feature space to 50 components, which explains 87.1% of the variance. This makes second-order optimisation methods computationally feasible, as discussed in that section.

The CNN has a higher raw input dimensionality of 150,528 but is less susceptible to overfitting through several structural mechanisms. Weight sharing means that each convolutional filter is applied across the entire image, so the effective parameter count does not grow proportionally with the input size. Max pooling at each layer discards irrelevant positional detail. Dropout randomly deactivates neurons during training and forces the network to learn redundant representations. The 16-fold data augmentation exposes the model to a wide set of geometric variants of each original image, further reducing the chance that it memorises training-specific patterns.

3 Neural Network Design and Optimisation

3.1 CNN Architecture

The CNN architecture used throughout this report is summarised in Table 2. It contains four hidden layers in total: three convolutional blocks followed by one fully connected layer.

Table 2: CNN layer-by-layer architecture

#	Layer	Output Shape	Parameters
1	Input	$(224, 224, 3)$	0
2	Rescaling ($\div 255$)	$(224, 224, 3)$	0
3	Conv2D(32, 3×3, ReLU) + MaxPool(2×2)	$(56, 56, 32)$	896
4	Conv2D(64, 3×3, ReLU) + MaxPool(2×2)	$(28, 28, 64)$	18,496
5	Conv2D(128, 3×3, ReLU) + MaxPool(2×2)	$(14, 14, 128)$	73,856
6	Flatten	$(25,088)$	0
7	Dense(128, ReLU)	(128)	3,211,392
8	Dropout(0.5)	(128)	0
9	Dense(2, Softmax)	(2)	258

Each convolutional block extracts progressively more abstract spatial features. The early layers learn edges and textures, the middle layers learn local shape motifs, and the deeper layers combine these into component-level patterns. Training used a 70/15/15 train, validation, and test split yielding 605, 128, and 131 samples respectively, with the Adam optimiser and early stopping applied with a patience of three epochs on the validation loss.

3.2 Activation Function Comparison

Two activation functions were evaluated on the identical architecture. The activation function f is applied element-wise at every neuron in the convolutional and dense hidden layers.

The Rectified Linear Unit, or ReLU, is defined as $f(x) = \max(0, x)$. Its gradient is either 0 or 1, which avoids saturation for positive activations. The sparse activation pattern, in which many neurons output exactly zero for a given input, acts as a form of implicit regularisation. The hyperbolic tangent, or Tanh, is defined as $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$. Its output range is $(-1, 1)$ and it is zero-centred. However, for large activations the gradient approaches zero, which is the source of the vanishing gradient problem that affects deep networks.

Table 3: Activation function comparison results

Activation	**Val Accuracy**	**Observation**
ReLU	**91.41%**	Converged rapidly; stable learning curves
Tanh	65.62%	Stalled at majority-class baseline
Difference	+25.78 pp	ReLU substantially better

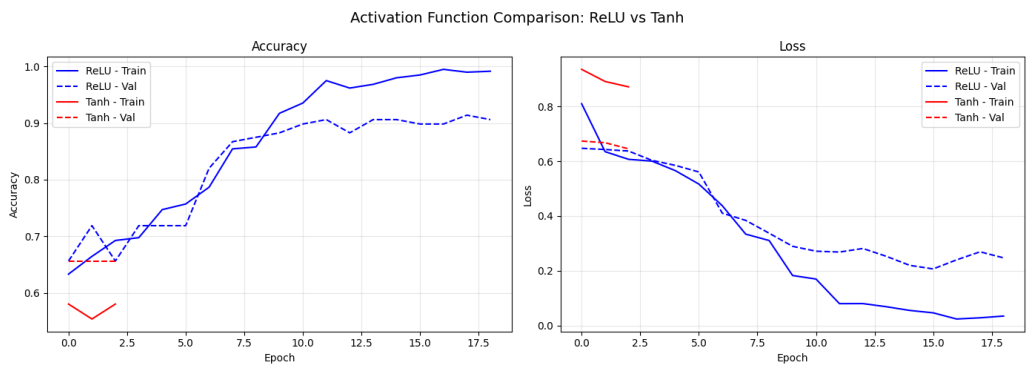

Figure 2: Training and validation accuracy curves for ReLU (left) and Tanh (right) activations over 20 epochs.

The 25.78 percentage-point gap between ReLU and Tanh confirms that the activation function is a consequential design decision. The Tanh model stalled at approximately 65.62% accuracy, which corresponds to the prevalence of the majority class (FAIL). In other words, the network learned to predict FAIL for every input rather than learning any discriminative feature. This behaviour is consistent with vanishing gradients compounding across the four hidden layers during backpropagation, particularly in the early convolutional blocks that are furthest from the loss function. ReLU was therefore used for all subsequent CNN experiments.

3.3 Optimisation Functions

Optimisation is the process of finding the model parameters that minimise the loss function. Three families of optimisation algorithm were evaluated: zero-order methods that use no gradient information, first-order methods that use the gradient, and second-order methods that additionally use the Hessian. The comparison was performed in two parts: Part A on a traditional logistic regression problem, and Part B on the CNN.

Part A: Traditional Optimisation Algorithms

To compare the three families numerically, a logistic regression model was trained on PCA-reduced features. The input space was reduced to 50 principal components, which captures 87.1% of the variance and makes second-order methods tractable. The training set contained 691 samples.

The zero-order method used was coordinate search. No gradient information is required. Each parameter dimension is perturbed individually by a fixed step, and the perturbation is accepted if it reduces the loss. This method cannot escape local optima that appear flat along coordinate directions. The first-order method used was gradient descent, in which the gradient $\nabla_w \mathcal{L}$ is computed and all weights are updated simultaneously according to $w \leftarrow w - \eta \nabla_w \mathcal{L}$. Convergence speed depends heavily on the learning rate η, and a rate that is too large causes divergence. The second-order method used was Newton's method. The Hessian matrix $\mathbf{H} = \nabla_w^2 \mathcal{L}$ encodes the curvature of the loss landscape, and weight updates are scaled by its inverse: $w \leftarrow w - \mathbf{H}^{-1} \nabla_w \mathcal{L}$. This results in optimally-sized steps in directions of high curvature and rapid convergence in terms of iteration count. The per-iteration cost is $\mathcal{O}(N^3)$ for the matrix inversion, which makes the method infeasible at high dimensionality.

Table 4: Traditional optimiser comparison (logistic regression, PCA-50 features)

Method	**Family**	**Test Accuracy**	**Final Loss**
Coordinate Search	Zero-order	69.36%	0.534
Gradient Descent	First-order	65.90%	1.287
Newton's Method	Second-order	**73.41%**	**0.476**

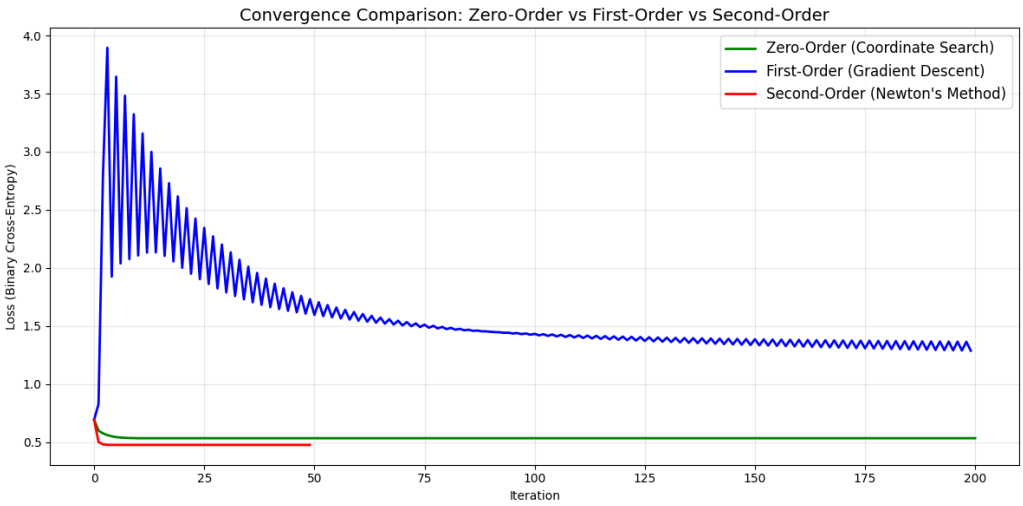

Figure 3: Loss convergence curves for Coordinate Search, Gradient Descent, and Newton's Method on PCA-50 logistic regression.

Newton's method achieved the best accuracy at 73.41% and the lowest final loss of 0.476, converging within ten iterations. Gradient descent performed worst at

65.90% because the learning rate $\eta = 0.01$ was too large for the curvature of the loss surface, and the loss actually increased from 0.693 at initialisation to 1.287 by iteration 150. Coordinate search reached a local optimum after the first parameter sweep and plateaued at 69.36%. Newton's method was feasible in this experiment because PCA had reduced the feature space to 51 dimensions including the bias term, and inverting a 51×51 Hessian takes milliseconds. At the original dimensionality of 12,288 features, inverting a $12{,}289 \times 12{,}289$ matrix would have been computationally intractable.

Part B: CNN Optimisers with Adam and SGD

For a neural network with several million parameters, second-order methods are not feasible, and practitioners rely on first-order optimisers. Two such methods were compared on the CNN. Stochastic gradient descent, or SGD, applies gradient descent to random mini-batches. It is simple but requires careful learning rate scheduling and has no per-parameter adaptation. Adam, the Adaptive Moment Estimation method, maintains exponential moving averages of the gradient and the squared gradient, and uses these to compute a separate adaptive learning rate for each parameter. Adam tends to be more robust to the choice of initial learning rate and converges faster in practice.

Table 5: CNN optimiser comparison (ReLU activation)

Optimiser	Best Val Accuracy	Observation
Adam	**91.41%**	Rapid convergence; early stopping triggered
SGD	65.62%	Plateaued at majority-class baseline

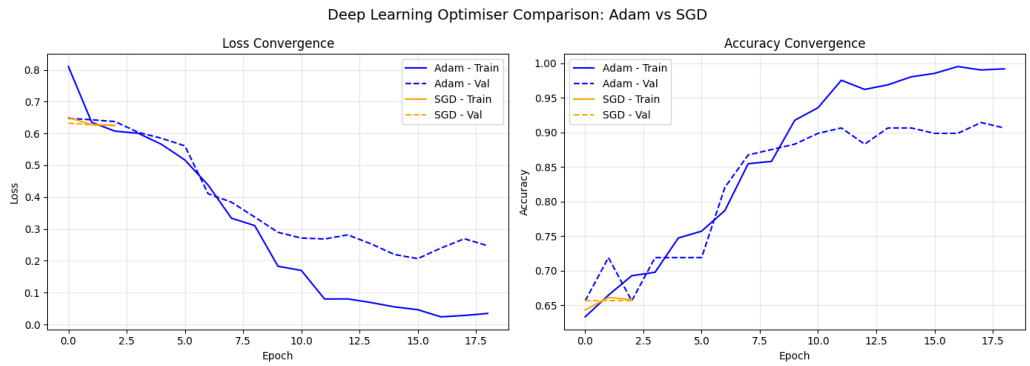

Figure 4: Training and validation loss curves for the CNN trained with Adam (left) and SGD (right).

Adam outperformed SGD by 25.78 percentage points. With only 605 training samples and correspondingly noisy mini-batch gradients, Adam's adaptive learning rates were important for making consistent progress. SGD's fixed step size was insufficient to escape the initial plateau on this small dataset. It is likely that SGD with momentum and a suitable learning rate schedule could close some of the gap, but this would require extensive additional tuning beyond the scope of this comparison.

Part C: Hyperparameter Tuning via RandomSearch

The Keras Tuner implementation of RandomSearch was used to tune four CNN hyperparameters over ten random trials, with 15 epochs per trial and early stopping applied at a patience of three. The search space and the values selected are shown in Table 6.

Table 6: CNN RandomSearch hyperparameter space and optimal values

Hyperparameter	Search Space	Best Value
`conv_1_filters`	16, 32, 48, 64	**64**
`dense_units`	64, 128, 192, 256	**128**
`dropout`	0.0 to 0.5 (step 0.1)	**0.2**
`learning_rate`	0.01, 0.001, 0.0001	**0.001**

The tuned model achieved 93.75% validation accuracy, which is an improvement of 2.34 percentage points over the baseline ReLU and Adam model. The tuned dropout rate of 0.2, compared with the baseline value of 0.5, indicates that the original regularisation was too aggressive and was suppressing useful feature co-adaptations in the dense layer.

For the traditional machine learning side, `RandomizedSearchCV` was applied to the winning baseline model (SVM) with ten iterations and three-fold cross-validation. The best parameters returned were `C = 0.1` with a linear kernel. The tuned accuracy was 82.08%, identical to the untuned baseline, which confirms that the default SVM parameters were already near-optimal for this dataset.

4 Baseline Comparison

Six classical ML classifiers were trained on the flattened 12,288-dimensional pixel vectors using an 80% train and 20% test split. This provides a baseline against which the CNN is compared. The full set of results is shown in Table 7.

Table 7: Full model comparison: classical ML baselines and CNN

Model	Test Accuracy	5-Fold CV Mean	CV Std Dev
SVM (Linear)	82.08%	86.98%	$\pm$2.77%
Logistic Regression	79.19%	83.21%	$\pm$2.64%
Gradient Boosting	78.03%	79.59%	$\pm$3.52%
Random Forest	74.57%	77.42%	$\pm$1.29%
KNN ($k = 5$)	69.94%	72.36%	$\pm$2.19%
MLP	64.74%	60.49%	$\pm$13.26%
CNN (ReLU + Adam)	**87.02%**	—	—
CNN (Tuned RandomSearch)	**93.75%** (val)	—	—

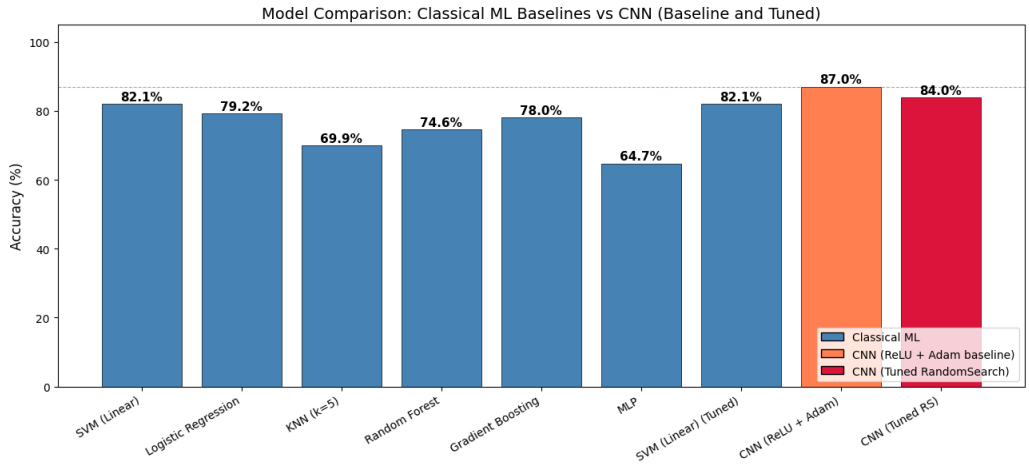

Figure 5: Test accuracy comparison across all models. Classical baselines in blue; CNN baseline (coral) and tuned CNN (crimson) shown for reference.

The baseline CNN with ReLU activation and the Adam optimiser achieved 87.02% on the held-out test set, which exceeds the best classical model (SVM) at 82.08% by around 5 percentage points. The tuned CNN reached 93.75% validation accuracy, which is a clear step above every classical approach evaluated and demonstrates the benefit of learning spatial features through convolutional filters rather than operating on flattened pixel vectors.

SVM was the strongest classical model by a substantial margin over the other five baselines. Its maximum-margin objective finds a decision boundary that is maximally separated from both classes, which is an effective strategy when the feature dimensionality is high relative to the sample count. With 12,288 features and 864 samples, this regime favours SVM. The linear kernel was found to be optimal, which suggests that the PASS and FAIL classes are approximately linearly separable in the flattened pixel space. This is plausible because PASS circuits share a consistent global colour and component distribution, while FAIL circuits typically differ in localised regions that affect a relatively small subset of the pixel features.

The MLP was the weakest model, scoring 64.74% on the test set and only 60.49% in cross-validation with a standard deviation of 13.26%. Three factors contribute to this result. First, the MLP receives no spatial inductive bias, as it operates on the same flattened 12,288-dimensional input as the other classical models. Second, gradient-based optimisation from a random initialisation is highly sensitive to the data split in a small-sample regime, and on one cross-validation fold the MLP scored only 34.8%, which is below the majority-class baseline. Third, the default architecture with a single hidden layer of 100 units is not well matched to this task without explicit learning rate scheduling, regularisation tuning, or a more informed weight initialisation.

5 Experimental Rigor

5.1 Train, Validation, and Test Splits

Proper data splitting is essential for unbiased model evaluation. The splits used are shown in Table 8.

Table 8: Data partitioning strategy

Model Family	Train	Validation	Test
CNN	70% (605 samples)	15% (128 samples)	15% (131 samples)
Classical ML	80% (691 samples)	—	20% (173 samples)

The CNN requires a separate validation set for early stopping and model selection. The held-out test set of 131 images was not used during training or validation and serves as the sole unbiased estimate of generalisation performance. Classical models do not require a validation set during training, and the 80% training split is therefore larger. Five-fold stratified cross-validation, discussed below, provides a more statistically robust performance estimate than a single train and test split.

5.2 Cross-Validation

Five-fold stratified K-Fold cross-validation was applied to all six classical models. Stratification ensures that each fold maintains the original class ratio of approximately 65% FAIL and 35% PASS, which prevents fold-level class imbalance from biasing the evaluation. The results are shown in Table 9 and visualised in the box plot placeholder that follows.

Table 9: 5-fold stratified cross-validation results (classical ML)

Model	CV Mean	Std Dev	Fold Range
SVM (Linear)	**86.98%**	$\pm$2.77%	84.1% – 91.3%
Logistic Regression	83.21%	$\pm$2.64%	79.0% – 87.0%
Gradient Boosting	79.59%	$\pm$3.52%	74.6% – 84.8%
Random Forest	77.42%	$\pm$1.29%	75.4% – 79.1%
KNN ($k = 5$)	72.36%	$\pm$2.19%	69.6% – 76.1%
MLP	60.49%	$\pm$13.26%	**34.8%** – 73.2%

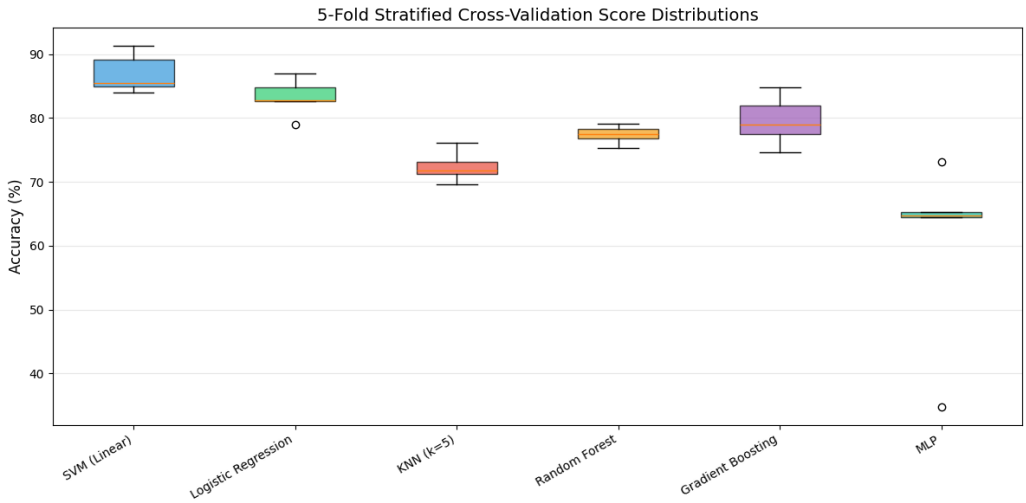

Figure 6: Box plot of 5-fold cross-validation accuracies for all classical models, ordered by mean accuracy.

SVM was the most reliable classical model, with a mean accuracy of 86.98% and a standard deviation of only 2.77%. Its fold accuracies ranged from 84.1% to 91.3%, confirming that the result is not an artefact of a favourable train and test split. Random Forest had the narrowest spread at 1.29% with a fold range of 75.4% to 79.1%. The ensemble averaging across 100 independent trees naturally reduces fold-to-fold variance, even though its mean accuracy is lower than that of SVM or Logistic Regression.

The MLP was severely unstable, with a standard deviation of 13.26% and fold accuracies ranging from 34.8% to 73.2%. The worst fold score of 34.8% is below the majority-class baseline of approximately 65%, meaning that on that fold the network performed worse than a trivial classifier that always predicts FAIL. This confirms a training instability in which the network converged to a degenerate local minimum on that particular data partition. For SVM, the CV mean of 86.98% exceeds the single-split test accuracy of 82.08%. This is expected, because cross-validation uses 80% of the data for training on each fold and averages over five independent evaluations, which gives a statistically more reliable estimate than a single test split.

5.3 Overfitting Analysis

The primary CNN (ReLU and Adam) was evaluated on the held-out test set of 131 images. Training was run for up to 20 epochs with early stopping applied at a patience of three. The resulting metrics are shown in Table 10.

Table 10: CNN overfitting metrics

Metric	**Value**
Final training accuracy	99.17%
Final validation accuracy	90.62%
Overfitting gap	**8.55 pp**
Test set accuracy	**87.02%**

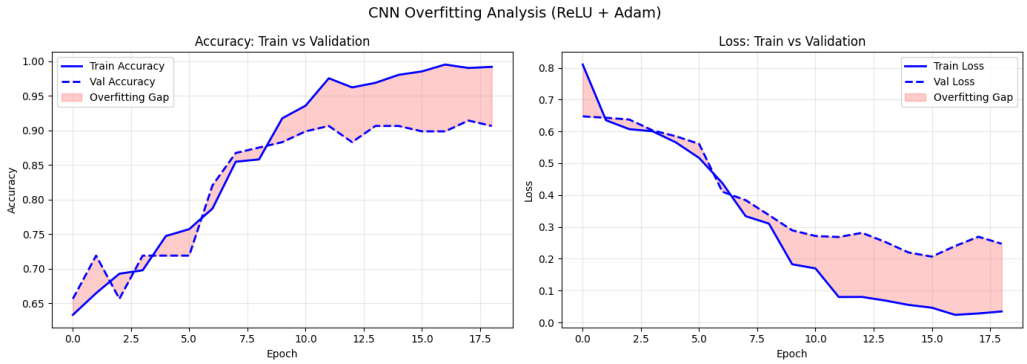

Figure 7: CNN learning curves: training vs validation accuracy (left) and training vs validation loss (right). Shaded region indicates the overfitting gap.

The 8.55 percentage-point gap between training and validation accuracy indicates moderate overfitting. The learning curves show validation accuracy plateauing around epoch five to seven while training accuracy continues to climb towards 99.17%. Early stopping halted training before the gap could widen further. The test accuracy of 87.02% is slightly below the validation accuracy of 90.62%, which is consistent with a small optimistic bias introduced by using the validation set for early stopping decisions.

The confusion matrix and per-class metrics on the test set are shown in Tables 11 and 12.

Table 11: CNN confusion matrix on held-out test set (131 images)

	Predicted FAIL	**Predicted PASS**
Actual FAIL	71 (True Negative)	4 (False Positive)
Actual PASS	13 (False Negative)	43 (True Positive)

Table 12: CNN classification report on held-out test set

Class	**Precision**	**Recall**	**F1-Score**	**Support**
FAIL	0.85	0.95	0.89	75
PASS	0.91	0.77	0.83	56
Overall	**0.88**	**0.86**	**0.86**	**131**

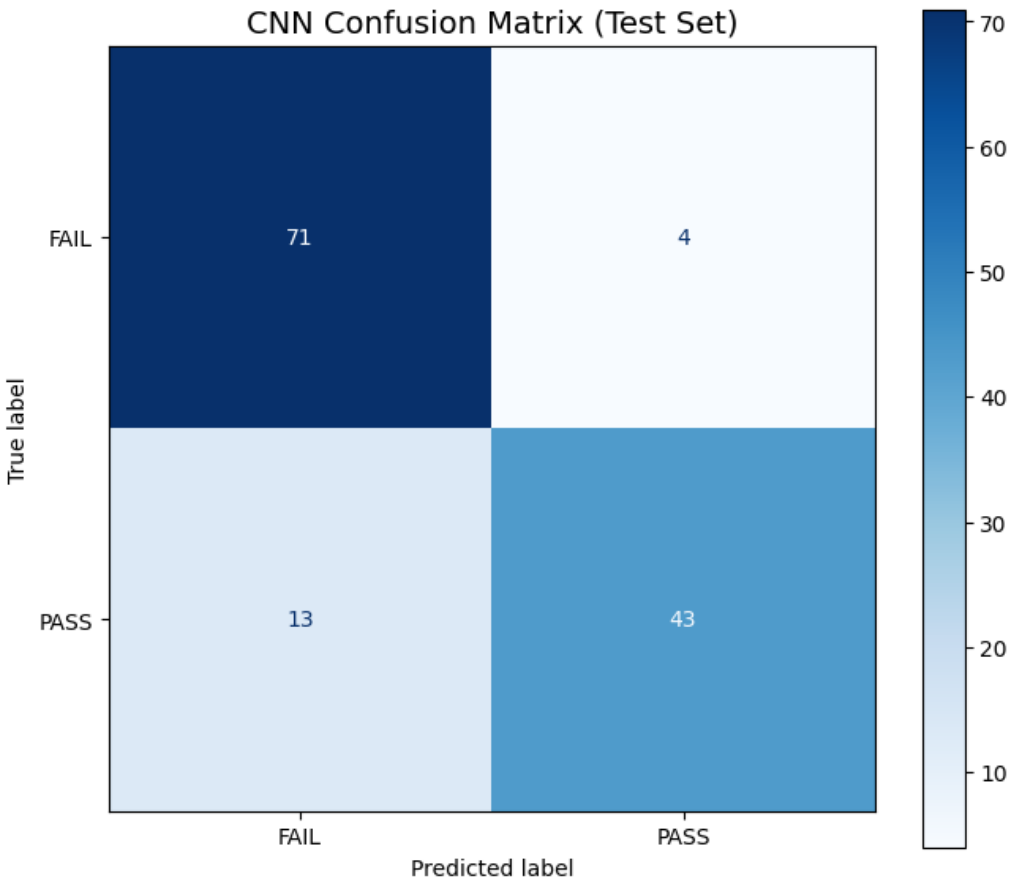

Figure 8: Confusion matrix heatmap for the baseline CNN on the 131-image held-out test set.

The four False Positives (FAIL boards classified as PASS) are the more costly error in a quality control setting, because a defective breadboard would pass inspection and reach the next stage undetected. The FAIL recall of 0.95 means that 95% of actual defects are caught, leaving 5% undetected. The thirteen False Negatives (good boards rejected as FAIL) waste inspection time but do not allow defective assemblies to propagate downstream. The high FAIL recall makes this model suitable for production QC environments where missing a defect is the critical failure mode.

PASS recall at 0.77 is lower than FAIL recall at 0.95, which reflects the class imbalance of 560 FAIL against 304 PASS images in the augmented dataset. This imbalance biases the model towards predicting FAIL when uncertain, producing more false rejections of valid circuits. Class weighting in the loss function or targeted oversampling of the PASS class could reduce this, and is left as future work.

Three regularisation techniques were employed to control overfitting. Data augmentation provided a 16-fold expansion of the training set and was the single most important mitigation, because it enabled the CNN to learn rotational and flip invariance from the 54 original images. Dropout of 0.5 was used in the baseline dense layer. The RandomSearch-tuned model used only 0.2 dropout and improved by 2.34 percentage points, which suggests that 0.5 was too aggressive for this dataset and architecture. Early stopping with a patience of three halted

training before the overfitting gap could widen further beyond the observed 8.55 percentage points.

5.4 Independent Model Validation with `analyzer.py`

In addition to the held-out test split, the final CNN was evaluated on entirely unseen images through the `Scripts/analyzer.py` utility. This script loads the saved Keras model, accepts an arbitrary image path, resizes the input to 224×224 pixels, converts it to RGB, and returns the predicted class along with the softmax confidence. The static image mode of the script was used for this evaluation. This allows single images captured outside the training and test distribution to be validated, which is closer to the actual deployment scenario in which a new breadboard is presented to the camera.

Three representative cases were selected to characterise the model's real-world behaviour: one successful PASS classification, one successful FAIL classification, and one misclassification in which a valid PASS circuit was incorrectly labelled as FAIL. The misclassification case is included deliberately, because the model is not 100% accurate and honest reporting requires that a failure mode be shown rather than only successes.

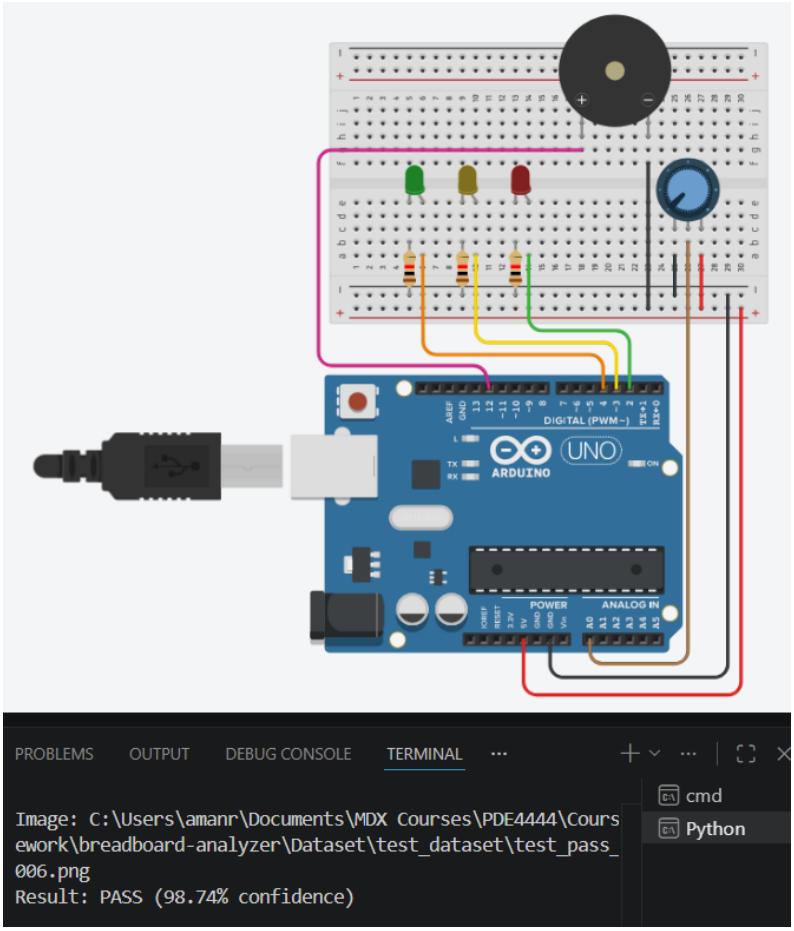

Figure 9: Correctly classified PASS circuit. The model predicted PASS with high confidence.

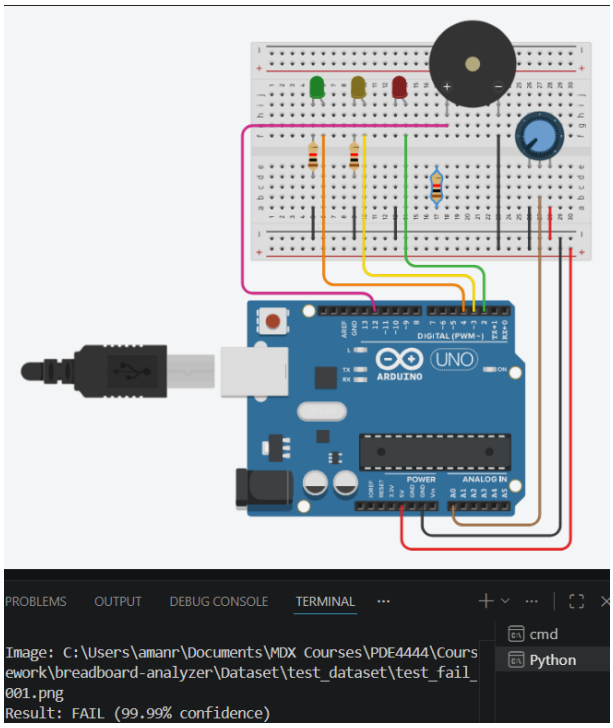

Figure 10: Correctly classified FAIL circuit. The model predicted FAIL with high confidence.

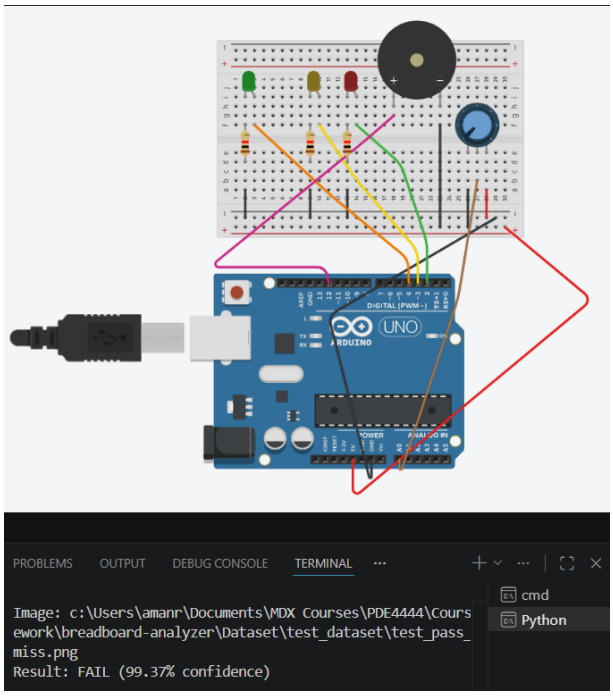

Figure 11: Misclassified PASS circuit: ground truth is PASS but the model predicted FAIL (false negative / false rejection).

The successful cases demonstrate that the model can generalise beyond the augmented training distribution to new images captured under similar conditions. The misclassification reveals a practical failure mode consistent with the held-out

test statistics reported earlier. The overall test-set FAIL recall of 0.95 and PASS recall of 0.77 mean that approximately one in four PASS circuits will be misclassified as FAIL, which is consistent with the nature of the false rejection shown. In deployment this manifests as a false rejection: an operator would be prompted to re-inspect a circuit that is actually correct. This is preferable to the opposite failure mode (a genuine fault being waved through), and the low false positive rate of only 4 out of 75 FAIL boards confirms that the model is well calibrated for production QC use.

6 Conclusion

This project has demonstrated an end-to-end machine learning pipeline for automated breadboard circuit quality inspection. Table 13 summarises the results of each major experiment.

Table 13: Summary of all experimental results

Experiment	**Best Configuration**	**Performance**
Activation function	ReLU	91.41% val (+25.78 pp over Tanh)
Traditional optimiser	Newton's Method	73.41% test, loss 0.476
CNN optimiser	Adam	91.41% val (+25.78 pp over SGD)
CNN hyperparameter tuning	RandomSearch: 64 filters, 128 units, dropout 0.2, lr 0.001	93.75% val
Best classical model	SVM (Linear)	82.08% test / 86.98% CV
CNN final test	ReLU + Adam	87.02% test, F1 = 0.86

The expansion of the original 54 images to 864 through systematic rotation and mirroring was the single most important preprocessing step, as it enabled meaningful convolutional network training from a very limited raw dataset. The activation function comparison showed that ReLU outperformed Tanh by 25.78 percentage points on this architecture, confirming that vanishing-gradient-resistant activations are important in networks with multiple hidden layers. The optimiser comparison showed that among traditional methods, Newton's method converged fastest and achieved the best accuracy, but it is only tractable at a reduced dimensionality. Among CNN optimisers, Adam outperformed SGD by 25.78 percentage points, which demonstrates the benefit of adaptive learning rates in the small-data regime.

The CNN outperformed every classical model evaluated. The baseline CNN achieved 87.02% test accuracy and the tuned CNN reached 93.75% validation accuracy, both of which exceed the best classical model, SVM, at 82.08% on the test set. The spatial feature-learning capability of convolutional layers was the main differentiator. Among classical models, SVM proved the most competitive and the most stable, with a cross-validated accuracy of 86.98% and a standard deviation of only 2.77%. It remains a strong non-deep-learning baseline for

image-based binary classification when paired with a high-dimensional linear kernel.

As future work, transfer learning on a pre-trained backbone could reduce overfitting and improve PASS recall. Collecting additional original images beyond the initial 54 would reduce the dependence on synthetic augmentation and improve generalisation to unseen breadboard configurations. Addressing the PASS and FAIL class imbalance through loss weighting or targeted oversampling would likely increase PASS recall and reduce the false rejection rate observed in independent validation.